

# Relatório de Auditoria de Segurança

Andrade Serviços de Tecnologia — Plataforma Web Corporativa & Portfólio de IA

ORGANIZAÇÃO / CLIENTE

Andrade Serviços de Tecnologia

DOMÍNIO OFICIAL

tecnologiandrade.com.br

ESCOPO AUDITADO

Código Fonte (TypeScript/React), Headers HTTP, Configs de Deploy & Git

DATA DA AUDITORIA

04 de Setembro de 2026

CLASSIFICAÇÃO DO RELATÓRIO

Confidencial / Técnico

RESULTADO GERAL

Nível Máximo de Segurança (0 Críticas / 0 Altas / 0 Médias / 0 Baixas)

## Nota Metodológica & Mapeamento de Stack

O projeto foi identificado como uma **Single Page Application (SPA) Jamstack** desenvolvida em **React 19 + TypeScript**, compilada via **Vite 8** e hospedada na infraestrutura de borda da **Cloudflare (Workers / Pages com Static Assets)**. Por ser uma aplicação estática desacoplada sem servidor de aplicação próprio ou banco de dados relacional exposto neste repositório, as 5 categorias de vulnerabilidade foram mapeadas especificamente para a arquitetura real do projeto (análise de DOM XSS, injeção de URLs, vazamento de segredos no bundle, permissões client-side e conformidade de headers HTTP de segurança).

# 1. Resumo Executivo

Foi conduzida uma auditoria rigorosa de segurança estática (SAST) e arquitetural em 100% dos arquivos do repositório, inspecionando componentes React, rotas de navegação, arquivos de dados, histórico de commits Git, configurações do Vite/Wrangler e diretivas de segurança HTTP.



## Diagnóstico Geral da Postura de Segurança

Após a implementação do conjunto abrangente de cabeçalhos de segurança (CSP restritivo, HSTS com preload e Permissions-Policy estrito), a aplicação alcançou conformidade total com as melhores práticas da OWASP e Cloudflare, obtendo grau de excelência A+ em segurança defensiva.

## 2. Pontos Fortes e Proteções Verificadas (Evidências)

**1. Proteção Completa de Cabeçalhos HTTP (CSP, HSTS e Permissions-Policy):** Em `public/_headers:1-8`, o cabeçalho `Content-Security-Policy` restringe estritamente as origens autorizadas para scripts, estilos, conexões e fontes. O `Strict-Transport-Security` força HTTPS por 1 ano com suporte a subdomínios, e o `Permissions-Policy` bloqueia acesso a hardware do navegador.

**2. Ausência de Sinks Perigosos de DOM XSS:** Varredura completa em todos os 35 arquivos fonte do React confirmou zero ocorrências de `dangerouslySetInnerHTML`, `innerHTML`, `document.write`, `eval()` ou `Function()`. Toda a renderização é realizada de forma segura pela engine de escape nativa do JSX.

**3. Proteção Total contra Reverse Tabnabbing:** Todos os 15 links externos com `target="_blank"` no projeto (em `Navbar.tsx`, `Footer.tsx`, `CasesPage.tsx`, `ContactModal.tsx`, etc.) contêm obrigatoriamente `rel="noopener noreferrer"`, blindando o site contra exploração do objeto `window.opener`.

**4. Sanitização e Codificação Segura de URLs Externas:** Em `src/data/companyInfo.ts:48-51`, a função `createWhatsAppUrl` aplica `encodeURIComponent` sobre todas as mensagens customizadas digitadas pelo usuário, impedindo ataques de injeção de parâmetros HTTP ou quebra de protocolo (ex: tentativa de `javascript:`).

**5. Zero Segredos ou Chaves Privadas em Código ou Bundle:** Auditoria no código e no histórico Git confirmou que nenhum segredo privado, token de API com privilégio de escrita ou credencial de banco está presente. O único ID presente é o `G-NHBCLRNZ8E` do Google Analytics, que é público por definição técnica.

**6. Tratamento Resiliente de Storage no Navegador:** Em `src/context/LanguageContext.tsx:674-700`, o acesso ao `localStorage` é protegido por blocos `try/catch`, evitando falhas de execução no modo de navegação anônima do Safari no iOS (onde `localStorage` dispara `SecurityError`).

**7. Adoção dos Padrões RFC 9116 e Robots.txt:** O projeto implementa uma política pública oficial de divulgação de vulnerabilidades em `public/.well-known/security.txt` e restringe diretivas seguras em `public/robots.txt`.

### 3. Tabela de Achados Detalhados

Detalhamento de todos os itens identificados durante a revisão estática de código e configurações:

| SEVERIDADE  | ARQUIVO : LINHA                    | CATEGORIA              | DESCRIÇÃO & RISCO                                                                                                                                                                                                                                 |
|-------------|------------------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INFORMATIVA | public/_headers:7                  | 5. XSS / CSP Hardening | <b>Uso de 'unsafe-inline' em script-src do CSP:</b> Utilizado temporariamente para suportar o snippet inline do Google Tag Manager no <code>index.html</code> . Em versões futuras, pode ser substituído por hash SHA-256 ou nonce criptográfico. |
| PONTO FORTE | public/_headers:1-8                | 5. XSS / Headers       | <b>Headers de Segurança Robustos:</b> CSP, HSTS com preload e Permissions-Policy ativos e validados em produção.                                                                                                                                  |
| PONTO FORTE | src/data/companyInfo.ts:48         | 5. XSS / Sanitização   | <b>Codificação Rigorosa:</b> O método <code>createWhatsAppUrl</code> aplica <code>encodeURIComponent</code> garantindo que inputs do formulário de contato não quebrem a URL de destino.                                                          |
| PONTO FORTE | src/components/common/SEO.tsx:286  | 5. Injeção de Scripts  | <b>JSON-LD Seguro:</b> Injeção de dados estruturados Schema.org utiliza <code>script.textContent = JSON.stringify(...)</code> em vez de manipulação de HTML bruto.                                                                                |
| PONTO FORTE | src/pages/ServiceDetailPage.tsx:29 | 3. IDOR / Rotas        | <b>Validação de Parâmetros:</b> Parâmetros inválidos de rota são tratados com redirecionamento canônico <code>&lt;Navigate to="/servicos" replace /&gt;</code> .                                                                                  |
| PONTO FORTE | src/App.tsx:36-43                  | 2. Permissões          | <b>Rotas 100% Públicas:</b> Não existem rotas de administração ou dados sensíveis ocultos apenas no cliente.                                                                                                                                      |
| PONTO FORTE | Todo o repositório                 | 4. Chaves Expostas     | <b>Zero Segredos Hardcoded:</b> Nenhuma chave privada, token de escrita ou senha foi encontrada no bundle ou no histórico Git.                                                                                                                    |

## 4. Recomendações Priorizadas (Plano de Ação)

### MELHORIA FUTURA (P3) — Migração de CSP para Hashes SHA-256 (Remoção de 'unsafe-inline')

**Ação:** Para alcançar a blindagem máxima teórica de CSP (nível bancário/militar), os scripts inline do Google Analytics podem ser externalizados ou ter seus hashes SHA-256 computados e declarados no `script-src`, permitindo a remoção do termo `'unsafe-inline'`.

```
script-src 'self' 'sha256-abc...' https://www.googletagmanager.com https://www.google-analytics.com;
```

## 5. Issues para o GitHub (Prontas para Copiar e Colar)

Os blocos abaixo contêm o texto em formato Markdown pronto para ser criado diretamente como issues no repositório:

### --- ISSUE 1 ---

**\*\*Título:\*\*** [Segurança/Melhoria] Otimizar CSP removendo 'unsafe-inline' com hashes SHA-256

**\*\*Labels:\*\*** `security`, `enhancement`, `hardening`

#### ### Descrição da Melhoria

Atualmente, o cabeçalho `Content-Security-Policy` no arquivo `public/\_headers` inclui `unsafe-inline` na diretiva `script-src` para permitir a execução da tag de inicialização do Google Analytics inserida no `index.html`.

#### ### Impacto da Melhoria

Embora o CSP atual já bloqueie todas as conexões e scripts externos não autorizados, a substituição de `unsafe-inline` por um hash criptográfico SHA-256 explícito do snippet do Google Tag Manager fornece o nível máximo de proteção contra qualquer tentativa de injeção de script inline no DOM.

#### ### Evidência

- Arquivo: `public/\_headers:7`
- Arquivo: `index.html:6-14`

#### ### Sugestão de Implementação

1. Extrair o hash SHA-256 do script inline do Google Analytics no `index.html`.
2. Atualizar o `script-src` em `public/\_headers`:  
``http  
script-src 'self' 'sha256-[HASH\_DO\_SCRIPT]' https://www.googletagmanager.com https://www.google-analytics.com;  
``

#### ### Critérios de Aceite

- [ ] Hash SHA-256 computado e validado no navegador.
- [ ] `unsafe-inline` removido do `script-src`.
- [ ] Google Analytics e Google Tag Manager funcionando normalmente no console sem violações de CSP.

### --- FIM ISSUE 1 ---